

Task 5: Mini Project — Student Performance Prediction

Dataset Used: Student Performance Dataset

Model Used: Decision Tree Classifier

Problem Type: Classification

Target: Pass / Fail

1. Introduction

The aim of this mini project is to build a simple Machine Learning model that predicts whether a student is likely to **Pass or Fail** based on their academic performance and study-related information.

A dataset containing **1,000 student records** was used for this project. The dataset includes study hours, attendance percentage, previous marks, assignment score, and the final Pass/Fail result.

A **Decision Tree Classifier** was selected because it is simple to understand and works well for classification problems. The model learns patterns from previous student data and uses those patterns to predict the result of new students.

[1]:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

[2]:

```
df = pd.read_csv("Student_Performance.csv")

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)

df.head()
```

Dataset loaded successfully!
Shape of dataset: (1000, 6)

[2]:

	Student_ID	Study_Hours	Attendance	Previous_Marks	Assignment_Score	Result
0	1	7	82	83	97	Pass
1	2	4	96	85	77	Pass
2	3	8	89	48	92	Pass
3	4	5	92	46	70	Pass
4	5	7	61	76	75	Pass

2. Data Cleaning

The dataset was checked for missing values using Pandas. Any missing records were removed during the cleaning process to ensure that the Machine Learning model received complete data.

After cleaning, the dataset was ready for feature and target selection.

[3]:

```
print("Dataset Information:")
print(df.info())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Student_ID            1000 non-null   int64  
 1   Study_Hours           1000 non-null   int64  
 2   Attendance            1000 non-null   int64  
 3   Previous_Marks        1000 non-null   int64  
 4   Assignment_Score      1000 non-null   int64  
 5   Result                1000 non-null   object  
dtypes: int64(5), object(1)
memory usage: 47.0+ KB
None
```

[4]:

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
Student_ID      0
Study_Hours     0
Attendance      0
Previous_Marks  0
Assignment_Score 0
Result          0
dtype: int64
```

3. Features and Target

The following features were selected:

- **Study Hours**
- **Attendance**
- **Previous Marks**
- **Assignment Score**

The target variable is **Result**, which contains two classes: **Pass** and **Fail**.

The Student_ID column was not used for prediction because it is only an identification number.

```
Assignment_Score    0
Result              0
dtype: int64
```

```
[6]:
```

```
features = [
    "Study_Hours",
    "Attendance",
    "Previous_Marks",
    "Assignment_Score"
]
```

```
X = df[features]
y = df["Result"]
```

```
print("Features:")
print(X.head())
```

```
print("\nTarget:")
print(y.head())
```

```
Features:
```

	Study_Hours	Attendance	Previous_Marks	Assignment_Score
0	7	82	83	97
1	4	96	85	77
2	8	89	48	92
3	5	92	46	70
4	7	61	76	75

```
Target:
```

0	Pass
1	Pass
2	Pass
3	Pass
4	Pass

```
Name: Result, dtype: object
```

4. Model Training

The dataset was divided into **80% training data** and **20% testing data**. The Decision Tree Classifier was trained using the training data.

The model learned relationships between study hours, attendance, previous marks, assignment scores, and student results.

```

1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 1 1 1 1 0 1 1 0 0 1 1 1 0 1 1 1 1
1 0 1 1 1 0 0 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 0 1 1 0 1 1 0 0 1
1 0 0 0 0 0 1 1 0 0 1 1 0 1 1 0 1 0 1 1 1 1 1 0 0 1 1 1 1 1 0 1 1 1 1 1
1 1 1 1 0 0 0 1 0 1 1 1 1 1 1 0 1 1 1 0 1 0 0 1 1 1 0 0 1 1 0 1 1 0 1 1 1
0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 0 1 0 1 0 1 0 1
1 0 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 1 0 1 1 0 1 1 1 1 1 1 1 0
0 0 1 1 0 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1 0 1 0 0 1 1 1
0 0 1 1 1 1 1 0 0 1 1 0 1 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 1 1 0 0 1 1 1 1 1 1
1 1 1 1 1 0 1 1 0 1 1 0 0 1 0 1 1 1 1 1 1 1 0 1 0 1 1 1 0 1 1 1 0 1 1 1 1
1 0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 1 1 1 0 1 1 1 1 0 0 1 1 0 0 1 1 1 1 1 1 1
1 1 1 0 1 0 1 0 0 1 1 0 1 1 1 1 0 1 1 1 0 0 1 1 1 1 1 1 1 0 1 0 0 1 0 0 1
1 1 1 1 1 0 1 1 0 1 0 1 0 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 0 1 0 1
1 0 1 1 0 1 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1
0 1 1 1 0 1 1 0 1 0 1 1 0 1 1 1 1 1 0 0 0 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 1
0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 0 1 1 1 0 0 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1
1 0 1 1 1 1 1 1 0 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1 1 1 0 1 1 1 0 1 1 1 1 0
1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 0 1 0 1 1 1 1 1 1 1 1 1 1 0
1]

```

Classes:
['Fail' 'Pass']

[8]:

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

```

```

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)

```

Training data shape: (800, 4)
Testing data shape: (200, 4)

[9]:

```

model = DecisionTreeClassifier(
    random_state=42,
    max_depth=3
)

print("Decision Tree Classifier created successfully!")

```

Decision Tree Classifier created successfully!

[10]:

```

model.fit(X_train, y_train)

print("Model trained successfully!")

```

Model trained successfully!

5. Results and Accuracy

After training, the model was used to predict the results of the testing data. The accuracy was calculated by comparing the predicted results with the actual results.

Model Accuracy: 89.5%

[11]:

```
y_pred = model.predict(X_test)

print("Predicted values:")
print(y_pred)
```

Predicted values:

```
[0 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1 0
 1 0 1 0 1 0 1 1 1 0 0 1 1 0 1 1 0 1 1 0 0 1 0 1 1 1 1 1 1 0 1 1 1 1
 0 1 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 0 0 1 1
 0 1 0 1 1 1 0 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 1 1 0 1 1
 1 1 1 1 0 1 0 1 0 1 0 0 1 1 0 0 1 1 0 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1
 1 1 1 1 1 0 1 0 0 1 1 0 1 1]
```

[12]:

```
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Model Accuracy (%)", accuracy * 100)
```

Model Accuracy: 0.895
Model Accuracy (%): 89.5

6. Testing New Students

The trained model was also tested using three new student inputs.

For example:

Study Hours: 6

Attendance: 85%

Previous Marks: 72

Assignment Score: 75

The model predicts whether the student is likely to **Pass or Fail** based on the patterns learned during training.

[13]:

```
new_students = pd.DataFrame({
    "Study_Hours": [6, 2, 8],
    "Attendance": [85, 60, 92],
    "Previous_Marks": [72, 45, 88],
    "Assignment_Score": [75, 50, 90]
})

predictions = model.predict(new_students)

new_students["Prediction"] = label_encoder.inverse_transform(predictions)

print("Predictions for New Students:")
print(new_students)
```

Predictions for New Students:

	Study_Hours	Attendance	Previous_Marks	Assignment_Score	Prediction
0	6	85	72	75	Pass
1	2	60	45	50	Fail
2	8	92	88	90	Pass

[14]:

```
student = pd.DataFrame({
    "Study_Hours": [6],
    "Attendance": [85],
    "Previous_Marks": [72],
    "Assignment_Score": [75]
})

prediction = model.predict(student)

result = label_encoder.inverse_transform(prediction)[0]

print("New Student Details:")
print(student)

print("\nPrediction:", result.upper())
```

New Student Details:

	Study_Hours	Attendance	Previous_Marks	Assignment_Score
0	6	85	72	75

Prediction: PASS

7. Limitations

The model has some limitations. Student performance can be affected by many factors that are not included in this dataset, such as motivation, health, learning environment, difficulty of subjects, and family or financial circumstances.

Also, the dataset is a simulated dataset, so its predictions may not represent real-world student performance perfectly.

8. Future Improvements

The project could be improved by using a larger and more diverse real-world dataset. Additional features such as previous attendance history, number of completed assignments, study consistency, exam scores, and participation could be included.

Different Machine Learning algorithms such as **Logistic Regression, Random Forest, or Support Vector Machine** could also be compared to determine which model provides better performance.

Conclusion

This mini project demonstrates how Machine Learning can be used to predict student performance. A Decision Tree Classifier was trained using study hours, attendance, previous marks, and assignment scores. The model was evaluated using test data and was also used to predict the results of new students.

The project demonstrates the complete Machine Learning workflow:

Dataset → Cleaning → Features & Target → Train-Test Split → Model Training → Prediction → Accuracy Evaluation